



Lokmanya Tilak Jankalyan Shikshan Sanstha's
PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR
(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)



DEPARTMENT OF INFORMATION TECHNOLOGY

CLOUD COMPUTING- LAB MANUAL

Subject code: 25UIT603T

Semester: VI



Lokmanya Tilak Jankalyan Shikshan Sanstha's
PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR
(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)



DEPARTMENT OF INFORMATION TECHNOLOGY

INDEX

Sr No.	Contents	Page No.
1	Vision & Mission of the Institute	1
2	Vision & Mission of the Department	2
3	Program Educational Objectives	2
4	Program Specific Outcomes	2
5	Program Outcomes	3
6	Course Outcomes & CO-PO & PSO Mapping	4
List of Practical's & Content Beyond Syllabus		
1	Practical No. 1	8
2	Practical No. 2	12
3	Practical No. 3	16
4	Practical No. 4	18
5	Practical No. 5	19
6	Practical No. 6	22
7	Practical No. 7	27
8	Practical No. 8	29
9	Practical No. 9	32
10	Practical No. 10	34





Lokmanya Tilak Jankalyan Shikshan Sanstha's
PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR
(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)



DEPARTMENT OF INFORMATION TECHNOLOGY



LOKMANYA TILAK JANKALYAN SHIKSHAN SANSTHA'S
PRIYADARSHINI COLLEGE OF ENGINEERING
An Autonomous Institute Affiliated to R.T.M. Nagpur University, Nagpur
Accredited with Grade 'A+' by NAAC
Near C.R.P.F. Campus, Hingna Road, Nagpur - 440 019 (Maharashtra) India
Phone : 07104 - 299648, Fax : 07104-299681
E-mail : principal.pce.ngp@gmail.com • Website: www.pcenagpur.edu.in
AICTE ID No. 1-5435581; DTE CODE No. 4123,
UNIVERSITY CODE No. : 278



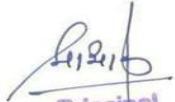
Vision

To become one of the India's leading Engineering Institutes in both education and research. We are committed to provide quality and state-of-the-art technical education to our students so that they become Technologically competent and in turn contribute for creating a great society.

Mission

1. Fostering a dynamic learning environment that equips students with Technical expertise, problem-solving skills and a deep commitment to ethical practices.
2. To cultivate a culture of innovation, incubation, research and entrepreneurship that drives technological advancements.
3. To uphold the spirit of mutual excellence while interacting with stake holders of our Institutional ecosystem.
4. Promoting lifelong learning, professional growth and ensuring holistic development of students and the well being of society.




Principal
Priyadarshini College of Engg.
Nagpur.

LOKMANYA TILAK JANKALYAN SHIKSHAN SANSTHA

Lokmanya Tilak Bhavan, Laxmi Nagar, Nagpur - 440 022, Maharashtra, INDIA. Tel : +91-712-2230665, 2245121. Fax No. : +91-712 2221430. Website : www.ltjss.net



Lokmanya Tilak Jankalyan Shikshan Sanstha's
PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR
(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)



DEPARTMENT OF INFORMATION TECHNOLOGY



LOKMANYA TILAK JANKALYAN SHIKSHAN SANSTHA'S
PRIYADARSHINI COLLEGE OF ENGINEERING
(Recognised by A.I.C.T.E. New Delhi & Govt. of Maharashtra Affiliated to R.T.M. Nagpur University)
Near C.R.P.F. Campus, Hingna Road, Nagpur - 440 019 (Maharashtra) India
Phone : 07104 - 299648, Fax : 07104-299681
E-mail : principal.pce.ngp@gmail.com • Website: www.pcenagpur.edu.in
AICTE ID No. 5435581; DTE CODE No. 4123; UNIVERSITY CODE No. 278
Accredited with Grade A+ by NAAC



Department of Information Technology

Date: 16/09/2023

Vision

To nurture the students with latest technology and research, helping them to become valuable contributors who can address the changing needs of the IT industry and society.

Mission

1. To create a holistic learning environment which empower the emerging graduates with advanced technical and value based education.
2. To attain professional excellence in the field of Information Technology and related areas.
3. To encourage transformative learning for research addressing the changing needs of society and fostering lifelong learning.

Bansod
H.O.D.
Department of Information Tech
Priyadarshini College of Engg
Nagpur-19

LOKMANYA TILAK JANKALYAN SHIKSHAN SANSTHA

Lokmanya Tilak Bhavan, Laxmi Nagar, Nagpur - 440 022, Maharashtra, INDIA. Tel : +91-712-2230665, 2245121, Fax No. : + 91-712 2221430. Website : www.ltjss.net



Lokmanya Tilak Jankalyan Shikshan Sanstha's
PRIYADARSHINI COLLEGE OF ENGINEERING, NAGPUR
(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)



DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAM OUTCOMES:

Engineering Graduates will be able to:

POs	Program Outcomes
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO6	The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO8	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO9	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO10	Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO11	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAM SPECIFIC OUTCOMES :

PSOs	Program Specific Outcomes
PSO1	An ability to apply mathematical foundations, algorithmic principles and computer science theory in the modeling and design of software systems of varying complexity.
PSO2	An ability to work with Open-Source Software and use off the shelf utilities for program integration.

COURSE OUTCOMES :

Course Outcomes	Statement
CO1	Configure various virtualization tools such as Virtual Box, VMware workstation.
CO2	Design and deploy a web application in a PaaS environment.
CO3	Install and use a generic cloud environment that can be used as a private cloud.

CO-PO & PSO MAPPING:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	3	2	2	1	1	1	1	2	1	1	3	3
CO2	3	3	3	2	2	1	1	1	1	2	1	1	3	3
CO3	3	3	3	2	2	1	1	1	1	2	1	1	3	2



List of Practical's:

Sr. No.	Name of Practical	CO
1	Use gcc to compile c-programs. Split the programs to different modules and create an application using make command.	CO1
2	Use version control systems command to clone, commit, push, fetch, pull, checkout, reset, and delete repositories.	CO1
3	Install Virtual-box/VMware Workstation with different flavours of linux or windows OS on top of windows 7 or 8.	CO2
4	Install a C compiler in the virtual machine created using virtual box and execute Simple Programs	CO2
5	Install Google App Engine. Create hello world app and other simple web applications using python/java.	CO2
6	Use GAE launcher to launch the web applications.	CO2
7	Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.	CO2
8	Find a procedure to transfer the files from one virtual machine to another virtual machine.	CO2
9	Find a procedure to launch virtual machine using trystack (Online Openstack Demo Version)	CO2
10	Install Hadoop single node cluster and run simple applications like wordcount.	CO3
Content Beyond Syllabus		
I1	Mini – Project	CO3



PRACTICAL NO.1

Aim: Use gcc to compile c-programs. Split the programs to different modules and create an application using make command.

Theory:

GCC:

A wholly-owned offshore unit of a multinational company providing specialized services like IT, R&D, finance, and HR, focusing on cost efficiency, talent access, and innovation. It can also refer to the

```
gcc -Iinclude -c src/main.c -o main.o
gcc -Iinclude -c src/math_utils.c -o math_utils.o
gcc main.o math_utils.o -o bin/app
```

```
project/
├── src/
│   ├── main.c
│   └── math_utils.c
├── include/
│   └── math_utils.h
├── Makefile
└── bin/
```

2. Header File (Interface)

Program:

include/math_utils.h

Copy code

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
```

```
int add(int a, int b);
int subtract(int a, int b);
```

```
#endif
```

3. Source Files (Implementation)

src/math_utils.c

Copy code

```
#include "math_utils.h"
```



```
int add(int a, int b) {  
    return a + b;  
}
```

```
int subtract(int a, int b) {  
    return a - b;  
}
```

src/main.c

c

Copy code

```
#include <stdio.h>
```

```
#include "math_utils.h"
```

```
int main()  
{
```

```
    int x = 10, y = 5;
```

```
    printf("Add: %d\n", add(x, y));
```

```
    printf("Subtract: %d\n", subtract(x, y));
```

```
    return 0;
```

```
}
```

4. Compiling Manually with gcc (Without Make)

This shows what make will automate.

bash

Copy code

```
gcc -Iinclude -c src/main.c -o main.o
```

```
gcc -Iinclude -c src/math_utils.c -o math_utils.o
```

```
gcc main.o math_utils.o -o bin/app
```



```
ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);
int subtract(int a, int b);
#endif
```

3. Source Files (Implementation)

src/math_utils.c

Copy code

```
#include "math_utils.h"
```

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

```
int subtract(int a, int b) {
```

```
    return a - b;
```

```
}
```

src/main.c

Copy code

```
#include <stdio.h>
```

```
#include "math_utils.h"
```

```
int main() {
```

```
    int x = 10, y = 5;
```

```
    printf("Add: %d\n", add(x, y));
```

```
    printf("Subtract: %d\n", subtract(x, y));
```

```
    return 0;
```

```
}
```

```
int add(int a, int b)
```

```
{
```



```
return a + b;  
}  
int subtract(int a, int b) {  
return a - b;
```

Viva Questions and answer

Q1 .What is cloud computing

Ans:-Cloud computing Saas,Paas,Aaas and provide online services on demand IT networking software.

Q2. What is Gcc?

Ans-GCC most commonly stands for Global cabability center a wholly-owned offshore unit of a multinational company providing specialized services.



PRACTICAL NO: 2

Aim: Use version control systems command to clone, commit, push, fetch, pull, checkout, reset, and delete repositories.

Theory:

Control system:

A control system is a set of mechanical or electrical devices that regulates other devices that regulates other devices or system by way of control loop

Element of Contro system-A typical contro system has the following four basic

Input signals,processing to be controlled,controller to process the input signals and regulate the

Process and output correspond to applied input.

Basic Definition

1.CLONE -Copies an existing remote repository (usually from GitHub/GitLab) to your local machine.

Creates a full local working copy with history.

2.COMMIT -Saves changes to the local repository with a message describing what was changed.

Think of it as taking a snapshot of your work.

3.PULL -Uploads local commits to a remote repository.

Shares your changes with others.

4. PUSH-Downloads changes from the remote repository but does **not** merge them into your current branch.

Lets you see what's new without affecting your work.

5. CHECKOUT-Fetches and automatically merges changes from the remote repository into your current branch.

Essentially: fetch + merge.

6. RESET-Moves the current branch pointer to a previous commit, optionally modifying the staging area and working directory.

7. DELETE-There is no single Git command to delete a repository.



1. **Local repo:** delete the project folder
2. **Remote repo:** delete it via the hosting service (e.g., GitHub settings)

Used to undo commits or unstage changes to move to another branch or revert files to a previous state.

For automobile driving system, the input signal is the force on accelerator pedal which through linkages causes the carburettor valve to open so as the engine brings the engine vehicle speed to desired value.



Algorithm:

1. Git helps developers **save versions of their code, work together**, and **go back to previous versions** if something goes wrong. Elements smaller than pivot go to the left.

Elements greater than pivot go to the right.

2. Keeps a **history of change of change in multiple computer**.

Program:

Basic Command:

```
git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"
git config --list
git clone <repository_url>
git status
git add filename
git commit -m "commit message"
git branch
git push
git checkout
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin <repo_url>
git push -u origin main
```

Output:

```
$ git status
On branch master

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)

    code1.c

nothing added to commit but untracked files present (use "git add" to track)
$ git add code1.c
$ git status
On branch master

Initial commit

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

    new file:   code1.c
```



Conclusion: We have Successfully execute the basic command of GIT

VIVA QUESTIONS:

1. What is Git and why is it used?

Ans: **Git** is a highly efficient **sTool/algorithm** based on the **tracking and Checking** strategy. It was developed by **Linux Torando** in 1960 and is widely used due to average cases application.

2. What is the difference between Git and Github?

Ans: Git is tool and Github is software utility eg. mericurial and azure, Devop.

2. What are repositories in Git? Explain local and remote repositories?

Ans: **Local repo:** delete the project folder

Remote repo: delete it via the hosting service (e.g., GitHub settings)



PRACTICAL NO. 3

Aim Install Virtual-box/VMware Workstation with different flavours of linux or windows OS on top of windows7 or 8

Theory:

What is a Virtual -box?

To install VirtualBox or VMware Workstation on Windows 7 / Windows 8, and then install different Linux or Windows OSes as virtual machines.

1. Prerequisites

Before starting, ensure:

- A. Windows 7 / 8 (64-bit preferred)
- B. Minimum 4 GB RAM (8 GB recommended)
- C. At least 20–40 GB free disk space
- D. Virtualization enabled in BIOS
- E. Intel: VT-x
- F. AMD: AMD-V

Check in Task Manager → Performance → CPU → Virtualization: Enabled

2. Choose a Virtualization Software

Option A: Oracle VirtualBox (Free)

2. Choose a Virtualization Software

Option A: Oracle VirtualBox (Free)

Supports Windows, Linux, macOS hosts

Ideal for students and labs

Option B: VMware Workstation

VMware Workstation Player – Free for personal use

VMware Workstation Pro – Paid, more advanced features

3. Install VirtualBox (on Windows 7/8)

- 1. Download VirtualBox for Windows host
 - 2. Double-click the installer
 - 3. Click Next → Next → Install
 - 4. Allow network driver installation
 - 5. Finish installation
 - 6. (Optional) Install Extension Pack (same version as VirtualBox)
- Install VMware Workstation (on Windows 7/8)
- 1. Download VMware Workstation Player or Pro
 - 2. Run the installer
 - 3. Accept license → Next
 - 4. Choose default options
 - 5. Install → Restart system if prompted

5. Download Guest OS ISO Files

Examples:

Linux Distributions

- ☐ Ubuntu (.iso)



- ☐ CentOS / Rocky Linux
- ☐ Fedora
- ☐ Kali Linux
- ☐ Linux Mint

Various Version of windows

Conclusion: We have Successfully Executed the install the virtual box in th different flavours of linux or windows OS.

Viva Question and Answers:

1. **What is Vmware Workstation?**

Ans: Vmware Workstation is a desktop virtualization software that allow a user to run multiple operating system as virtual machine on a single physical computer.

2. **What is type of hypervisor is Vmware Workstation ?**

Ans:It is a Type-2(hosted)hypervisor because it run on top of a hos to operating system.

3. **How is a stack implemented using an array?**

Ans: Feature of Vmware workstation:Multiple OS support
Snapshots and cloning.



PRACTICAL NO. 4

Aim: Install a C compiler in Virtual machine created using virtual box and execute simple Program

Theory:

A **Virtual Machine (VM)** allows us to run an operating system inside another operating system. **GCC (GNU Compiler Collection)** is a widely used compiler for compiling C programs in Linux environments.

Program:

Open Terminal inside Ubuntu and run:

```
sudo apt update  
sudo apt upgrade
```

Step 4: Install GCC Compiler

Run the following command:

```
sudo apt install build-essential
```

Verify GCC installation:

```
gcc --version
```

Program to Print Sum of Two Numbers

Source Code (add.c)

```
#include <stdio.h>  
  
int main() {  
    int a, b, sum;  
    a = 10;  
    b = 20;  
    sum = a + b;  
    printf("Sum = %d", sum);  
    return 0;  
}
```

Steps to Execute the Program

1.Create a C file:

```
nano add.c
```

2.Compile the program:

```
gcc add.c -o add
```

3.Execute the program:

```
./add
```

Output

Sum = 30

Conclusion: We have Successfully install the C compiler in Virtual machine created using virtual box and execute simple Program



PRACTICAL NO: 5

Aim: Install Google App Engine .Create hello world app and other simple web application using python/java.

Theory:Google App Engine is a **Platform as a Service (PaaS)** that lets you deploy web applications without managing servers.

To use it, you must install **Google Cloud SDK**.

Operations Performed

Step 1: Create a Google Cloud Account

- A. Go to <https://cloud.google.com/>
- B. Sign in with your Google account
- C. Create a **new project**
 - o Click **Select Project** → **New Project**
 - o Give it a name (e.g., hello-app-engine)

Step 2: Install Google Cloud SDK

For Windows

1. Download installer from:
<https://cloud.google.com/sdk/docs/install>
2. Run the installer
3. During installation:
 - o Add gcloud to PATH
 - o Install Python (if prompted)
4. Finish installation

Verify installation

Open **Command Prompt** and run:

gcloud version

Step 3: Initialize Google Cloud SDK

Run:

gcloud init

Steps:

- Login to Google account
- Select your project
- Choose default region (you can press Enter)



Step 4: Install App Engine Components

For **Python**:

```
gcloud components install app-engine-python
```

For **Java**:

```
gcloud components install app-engine-java
```

Step 5: Create a Hello World App (Python)

Project Structure

```
hello-app/  
├── app.yaml  
├── main.py  
└── requirements.txt  
app.yaml  
runtime: python39  
main.py  
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return "Hello World from Google App Engine!"  
  
if __name__ == '__main__':  
    app.run()
```

requirements.txt

```
flask
```

Step 6: Deploy to App Engine

Go to your project folder:

```
cd hello-app
```

Deploy:

```
gcloud app deploy
```

When asked: Choose region (e.g., asia-south1) Type **Y** to continue

Step 7: Open Your App

gcloud app browse:

```
Hello World from Google App Engine!
```

Conclusion: We have Successfully install google app engine and create simple hello world using java

Viva Questions and Answers:

1. What is google apps engine and example?

Ans: Google is a platform as a service which deploys web applications without managing servers.

2. Which model is used in deploying applications on GAE?

Ans: Deployment model.



PRACTICAL NO: 6

Aim: Use GAE launcher to launch the web application.

Theory:

Install GAE Launcher

GAE Launcher is **deprecated** and no longer maintained, but it still works for legacy apps.

install **Google App Engine SDK (Python – legacy)**, which includes **GAE Launcher**.

GAE Launcher was mainly available for **Windows and macOS**.

Open GAE Launcher

Launch **GoogleAppEngineLauncher** from your applications menu.

Add Your Web Application

GAE Launcher is obsolete. For current projects, Google recommends using the **gcloud CLI**:

`gcloud app run`

or for Python:

`dev_appserver.py app.yaml`

This is the supported path going forward.

Conclusion: We have Successfully to launch the web application.

Viva Questions and Answers:

1. What is google app engine?

Ans: A google apps Engine is a platform as-services and deploy scalable application on cloud .It provides a platform for developer to build and different application on google cloud platforms

2. What are application of google app engine?

Ans-Google App Engine provide built –in services and API used databases

3. What are type of cloud ?

Ans: Cloud type:

- 1.Software as services.
- 2.Platform as services.
- 3.Infrastructure as services.



PRACTICAL NO: 7

Aim : Simulate a cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.

Theory:

CloudSim includes:

VM scheduling: TimeShared, SpaceShared, Task (Cloudlet) scheduling: **simple FCFS** inside DatacenterBroker.

2. Scenario Description

We will simulate:

1 Datacenter

3 VMs (different MIPS)

6 Cloudlets (different lengths)

Custom Min–Min Scheduling Algorithm

Min–Min idea:

1. For each cloudlet, estimate completion time on all VMs
2. Assign the cloudlet with the **minimum earliest completion time**
3. Update VM availability
4. Repeat until all cloudlets are assigned

3. Project Setup

Add CloudSim JARs:

cloudsim-3.0.3.jar

commons-math3.jar

4. Custom Broker with Min–Min Scheduling

Custom Broker Class

```
import org.cloudbus.cloudsim.*;
```

```
import org.cloudbus.cloudsim.core.CloudSim;
```

```
import java.util.*;
```

```
public class MinMinBroker extends DatacenterBroker
{
    public MinMinBroker(String name) throws Exception {
        super(name);
    }
}
```

Override

```
protected void submitCloudlets() {
```

```
    List<Cloudlet> cloudletList = new ArrayList<>(getCloudletList());
```

```
    List<Vm> vmList = new ArrayList<>(getVmList());
```



```
Map<Integer, Double> vmReadyTime = new HashMap<>();
for (Vm vm : vmList) {
    vmReadyTime.put(vm.getId(), 0.0);
}

while (!cloudletList.isEmpty()) {

    Cloudlet selectedCloudlet = null;
    Vm selectedVm = null;
    double minCompletionTime = Double.MAX_VALUE;

    for (Cloudlet cloudlet : cloudletList) {
        for (Vm vm : vmList) {

            double executionTime =
                cloudlet.getCloudletLength() / vm.getMips();
            double completionTime =
                vmReadyTime.get(vm.getId()) + executionTime;

            if (completionTime < minCompletionTime) {
                minCompletionTime = completionTime;
                selectedCloudlet = cloudlet;
                selectedVm = vm;
            }
        }
    }

    bindCloudletToVm(selectedCloudlet.getCloudletId(), selectedVm.getId());

    double execTime =
        selectedCloudlet.getCloudletLength() / selectedVm.getMips();
    vmReadyTime.put(
        selectedVm.getId(),
        vmReadyTime.get(selectedVm.getId()) + execTime
    );

    cloudletList.remove(selectedCloudlet);
}

super.submitCloudlets();
}
```

5. Main Simulation Class

```
import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;

import java.util.*;

public class MinMinSimulation {

    public static void main(String[] args) {
```



```
try {
    int numUsers = 1;
    Calendar calendar = Calendar.getInstance();
    CloudSim.init(numUsers, calendar, false);

    Datacenter datacenter = createDatacenter("Datacenter_1");

    MinMinBroker broker = new MinMinBroker("MinMinBroker");
    int brokerId = broker.getId();

    List<Vm> vmList = createVMs(brokerId);
    broker.submitVmList(vmList);

    List<Cloudlet> cloudletList = createCloudlets(brokerId);
    broker.submitCloudletList(cloudletList);

    CloudSim.startSimulation();
    CloudSim.stopSimulation();

    List<Cloudlet> result = broker.getCloudletReceivedList();
    printResults(result);

} catch (Exception e) {
    e.printStackTrace();
}
```

6. Datacenter, VM, and Cloudlet Creation

Datacenter

```
private static Datacenter createDatacenter(String name) {

    List<Host> hostList = new ArrayList<>();

    List<Pe> peList = new ArrayList<>();
    peList.add(new Pe(0, new PeProvisionerSimple(10000)));

    Host host = new Host(
        0,
        new RamProvisionerSimple(16384),
        new BwProvisionerSimple(100000),
        1000000,
        peList,
        new VmSchedulerTimeShared(peList)
    );

    hostList.add(host);

    DatacenterCharacteristics characteristics =
        new DatacenterCharacteristics(
            "x86", "Linux", "Xen",
            hostList, 10.0, 3.0, 0.05, 0.1, 0.1
        );
}
```



```
try {  
    return new Datacenter(name, characteristics,  
        new VmAllocationPolicySimple(hostList),  
        new LinkedList<>(), 0);  
} catch (Exception e) {  
    return null;  
}  
}
```

VMs

```
private static List<Vm> createVMs(int brokerId) {  
    List<Vm> vmList = new ArrayList<>();  
  
    vmList.add(new Vm(0, brokerId, 1000, 1, 2048, 1000, 10000, "Xen", new  
        CloudletSchedulerTimeShared()));  
    vmList.add(new Vm(1, brokerId, 2000, 1, 2048, 1000, 10000, "Xen", new  
        CloudletSchedulerTimeShared()));  
    vmList.add(new Vm(2, brokerId, 3000, 1, 2048, 1000, 10000, "Xen", new  
        CloudletSchedulerTimeShared()));  
  
    return vmList;  
}
```

Cloudlets

```
private static List<Cloudlet> createCloudlets(int brokerId) {  
  
    List<Cloudlet> cloudletList = new ArrayList<>();  
    UtilizationModel um = new UtilizationModelFull();  
  
    long[] lengths = {20000, 30000, 40000, 10000, 25000, 15000};  
  
    for (int i = 0; i < lengths.length; i++) {  
        Cloudlet cloudlet = new Cloudlet(  
            i, lengths[i], 1, 300, 300, um, um, um  
        );  
        cloudlet.setUserId(brokerId);  
        cloudletList.add(cloudlet);  
    }  
  
    return cloudletList;  
}  
  
private static void printResults(List<Cloudlet> list) {  
  
    System.out.println("Cloudlet ID | VM ID | Start | Finish");  
  
    for (Cloudlet c : list) {  
        System.out.println(  
            c.getCloudletId() + "\t\t" +  
            c.getVmId() + "\t" +  
            c.getExecStartTime() + "\t" +  
            c.getFinishTime()  
        );  
    }  
}
```



}

Conclusion: We have Successfully simulate the cloud scenario using CloudSim and run a scheduling algorithm that is not present in CloudSim.

Viva Questions and Answers:

1. What is CloudSim?

Ans CloudSim includes:

VM scheduling: TimeShared, SpaceShared, Task (Cloudlet) scheduling: **simple FCFS** inside

2. Why do simulates fix cloudSim?

Ans: Cloud Computing.

‘
-



PRACTICAL NO: 8

Aim: Find a procedure to transfer the files from one virtual machine using trystack(Online Openstack Demo Version)

Theory:

1. Requirement of System

TryStack account

Register at the TryStack site and wait for approval

OpenStack credentials

Method 1: Launch a VM using the Web Dashboard (Horizon)

Step 1: Log in

1. Open the **Horizon dashboard URL** provided by TryStack

Enter

Username

Password

Project

Domain (usually Default)

Step 2: Navigate to Instances

Go to:

Project → Compute → Instances

Step 3: Launch Instance

Click **Launch Instance**

Fill in the tabs:

Details

Instance Name: e.g. test-vm

Description: optional

Count: 1

Source

Boot Source: **Image**

Select an image (e.g. Ubuntu 20.04, Cirros)

Click the ↑ arrow to allocate it

Flavor

Choose a small flavor (e.g. m1.small, m1.tiny)

Network

Select the available private network (usually only one)



Key Pair

Select an existing key pair

OR

Create a new one and download the .pem file

Security Group

Ensure `default` is selected

(Later you can add SSH/ICMP rules)

Step 4: Launch

Click **Launch Instance**

VM status should move from **BUILD** → **ACTIVITY**

Step 5: Assign Floating IP (for SSH access)

1. Go to **Instances**
2. Click **Actions** → **Associate Floating IP**
3. Allocate a new IP (if needed)
4. Associate it with your instance

Step 6: Access the VM

```
chmod 600 key.pem
```

```
ssh -i key.pem ubuntu@<floating-ip>
```

(User may be `ubuntu`, `centos`, or `cirros` depending on image)

Method 2: Launch a VM using OpenStack CLI

Step 1: Download OpenStack RC file

In Horizon:

Project → **API Access** → **Download OpenStack RC File**

Source it:

```
source project-openrc.sh
```

Step 2: Verify resources

```
openstack image list
```

```
openstack flavor list
```

```
openstack network list
```

```
openstack keypair list
```

Step 3: Create a key pair (if needed)

```
openstack keypair create mykey > mykey.pem  
chmod 600 mykey.pem
```



Step 4: Launch the instance

```
openstack server create \  
  --image "Ubuntu 20.04" \  
  --flavor m1.small \  
  --network private \  
  --key-name mykey \  
  test-vm
```

Step 5: Allocate and assign Floating IP

```
openstack floating ip create public  
openstack server add floating ip test-vm <floating-ip>
```

Step 6: SSH into the VM

```
ssh -i mykey.pem ubuntu@<floating-ip>
```

Method 1: Launch a VM using the Web Dashboard (Horizon)

Step 1: Log in

Open the **Horizon dashboard URL** provided by TryStack.

Enter
Username
Password
Project
Domain (usually Default)

Step 2: Navigate to Instances

Go to:
Project → Compute → Instances

Step 3: Launch Instance

Click **Launch Instance**
Fill in the tabs:

Details
Instance Name: e.g. test-vm
Description: optional
Count: 1

Source
Boot Source: **Image**
Select an image (e.g. Ubuntu 20.04, Cirros)
Click the ↑ arrow to allocate it



Flavor

Choose a small flavor (e.g. `m1.small`, `m1.tiny`)

Networks:

Select the available private network (usually only one)

Select an existing key pair

OR

Create a new one and download the `.pem` file

Security Group

Ensure `default` is selected

(Later you can add SSH/ICMP rules)

Step 4: Launch

Click **Launch Instance**

VM status should move from **BUILD** → **ACTIVE**

Step 5: Assign Floating IP (for SSH access)

Go to **Instances**

Click **Actions** → **Associate Floating IP**

Allocate a new IP

Associate it with your instance

Step 6: Access the VM

```
chmod 600 key.pem
```

```
ssh -i key.pem ubuntu@<floating-ip>
```

(User may be `ubuntu`, `centos`, or `cirros` depending on image)

Method 2: Launch a VM using OpenStack CLI

Step 1: Download OpenStack RC file

In Horizon:

Project → **API Access** → **Download OpenStack RC File**

Source it:

```
source project-openrc.sh
```

Step 2: Verify resources

```
openstack image list
```

```
openstack flavor list
```

```
openstack network list
```

```
openstack keypair list
```



Step 3: Create a key pair (if needed)

```
openstack keypair create mykey > mykey.pem  
chmod 600 mykey.pem
```

Step 4: Launch the instance

```
openstack server create \  
  --image "Ubuntu 20.04" \  
  --flavor ml.small \  
  --network private \  
  --key-name mykey \  
  test-vm
```

Step 5: Allocate and assign Floating IP

```
openstack floating ip create public  
openstack server add floating ip test-vm <floating-ip>
```

Step 6: SSH into the VM

```
ssh -i mykey.pem ubuntu@<floating-ip>
```

Conclusion: We have Successfully Executed the Vmware using java program.

Viva Questions and Answers:

1. What is a VMware?

Ans: A VMware is form in which each node has at most two children, called the left child and the right child.

2. What are openstack?

Ans: Openstack is tree like structure tree, the child is always less than the parent, have the priority depend on application in java



PRACTICAL NO: 9

Aim: Find the procedure to transfer the file from one virtual machine to another virtual machine

Theory: Virtual machine

is a software-based emulation of a physical computer that runs its own operating system and applications, all sharing the resources (CPU, RAM, storage) of a single physical server using VMware's hypervisor software, allowing multiple isolated "computers" to run simultaneously on one machine for efficiency, testing, and flexibility.

Step to be follow :

1. Open **VirtualBox Manager**
2. Select **VM → Settings → Shared Folders**
3. Click **Add New Folder**
4. Choose:
 - **Folder Path:** Any folder on Host
 - **Folder Name:** e.g., shared
 - ☒ Auto-mount
 - ☒ Make Permanent
5. Click **OK**

Inside Both Virtual Machines

Tools

- **FileZilla**
- **WinSCP**

Steps

1. Open FileZilla in VM-1 or Host
2. Use:
 - Host: VM IP
 - Username & Password
 - Protocol: SFTP
 - Port: 22
3. Drag and drop files

Method 4: USB Drive (Manual but Works)

1. Attach USB to **VM-1**
2. Copy file to USB
3. Detach USB
4. Attach USB to **VM-2**
5. Copy files



Method 5: Using Network Folder (Advanced)

- Set both VMs to **Bridged Adapter**
- Use **NFS / Samba**
- Suitable for large networks

Which Method Should You Use?

Scenario	Best Method
Beginner / Practical Lab	Shared Folder
Linux Command Line	SCP
GUI	FileZilla
No Network	USB

If you tell me:

- **OS of both VMs** (Ubuntu, CentOS, Windows)
- **Host OS** (Windows/Linux)

Conclusion: We have Successfully transfer a file from one machine to another.

Viva Questions and Answers:

1. What is a Virtual Machine?

Ans: Virtual Machine (VM) is a software-based computer that runs an operating system inside a physical machine using virtualization software.

2. Why do we need to transfer files between virtual machines?

Ans: To share data, programs, source code, or configuration files between different virtual machines for testing, development, or practical work.



PRACTICAL NO: 10

Aim: To install a Hadoop Single-Node (Pseudo-Distributed) Cluster and run a WordCount .

Theory:

This is the most common setup for learning and testing Hadoop.

Prerequisites System

Linux (Ubuntu 20.04/22.04 recommended)

Minimum 4 GB RAM

User with sudo privileges

Program

1.Install the software

Hadoop requires Java 8 or 11.

```
sudo apt update
```

```
sudo apt install openjdk-11-jdk -y
```

Verify:

```
java -version
```

Set JAVA_HOME:

```
readlink -f /usr/bin/java | sed "s:bin/java::"
```

Add to ~/.bashrc:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

```
export PATH=$PATH:$JAVA_HOME/bin
```

Reload:

```
source ~/.bashrc
```

2. Create Hadoop User

```
sudo adduser hadoop
```

```
sudo usermod -aG sudo hadoop
```

```
su - hadoop
```

3. Configure SSH (Required by Hadoop)

```
sudo apt install openssh-server -y
```

```
ssh-keygen -t rsa -P ""
```

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

```
ssh localhost
```

4. Download and Install Hadoop

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
```

```
tar -xvzf hadoop-3.3.6.tar.gz
```

```
sudo mv hadoop-3.3.6 /usr/local/hadoop
```

```
sudo chown -R hadoop:hadoop /usr/local/hadoop
```

Add Hadoop environment variables to ~/.bashrc:

```
export HADOOP_HOME=/usr/local/hadoop
```

```
export HADOOP_INSTALL=$HADOOP_HOME
```

```
export HADOOP_MAPRED_HOME=$HADOOP_HOME
```

```
export HADOOP_COMMON_HOME=$HADOOP_HOME
```

```
export HADOOP_HDFS_HOME=$HADOOP_HOME
```

```
export YARN_HOME=$HADOOP_HOME
```

```
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
```

```
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
```



Reload:
source ~/.bashrc

5. Hadoop Configuration (Single Node)

5.1 hadoop-env.sh

Edit:
nano \$HADOOP_HOME/etc/hadoop/hadoop-env.sh
Set:
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

5.2 core-site.xml

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

5.3 hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
</configuration>
```

5.4 mapred-site.xml

```
cp mapred-site.xml.template mapred-site.xml
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

5.5 yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
```

hdfs namenode -format

7. Start Hadoop Services

bash
Copy code
start-dfs.sh
start-yarn.sh
Verify:

bash



DEPARTMENT OF INFORMATION TECHNOLOGY

Copy code

jps

Expected:

NameNode

DataNode

SecondaryNameNode

ResourceManager

NodeManager

Hadoop Web Interfaces

NameNode: <http://localhost:9870>

ResourceManager: <http://localhost:8088>

Run WordCount Example

Create Input File

bash

Copy code

mkdir ~/input

nano ~/input/data.txt

Example content:

nginx

Copy code

hello hadoop

hello world

hadoop world hadoop

9.2 Copy Data to HDFS

bash

Copy code

hdfs dfs -mkdir /input

hdfs dfs -put ~/input/data.txt /input

Verify:

bash

Copy code

hdfs dfs -ls /input

9.3 Run WordCount

bash

Copy code

hadoop jar \$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar \
wordcount /input /output

Output : hdfs dfs -cat /output/part-r-00000

Sample output:

Code:

Stop Hadoop

bash

Copy code



```
stop-yarn.sh  
stop-dfs.sh
```

Output:

```
hadoop 3  
hello 2  
world 2
```

Conclusion: We have Successfully install a Hadoop Single-Node (Pseudo-Distributed) Cluster and run a WordCount.

Viva Questions and Answers:

1. What is Hadoop?

Ans: Hadoop is open source framework for storing and processing massive datasets across cluster of commodity computer.

2. What is YARN?

Ans: Yet another resource negotiator based on hadoop or big data. Its simple programming model like Mapreduce to process data in parallel and breaking large datasets in small.

Content beyond Syllabus
PRACTICAL NO: I1

Aim: Mini project